

题目链接

本题是2009年ICPC亚洲区域赛东京赛区的E题

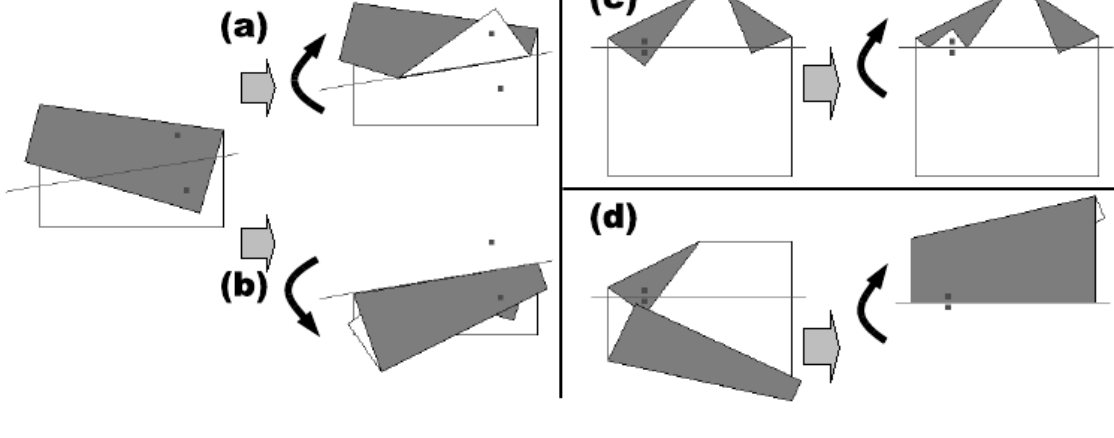
题意

有一个100mm×100mm的纸片，进行若干次折叠操作：每次给出点P、Q，沿着P、Q的垂直平分线将包含P的部分翻折至P点在Q点上。最后给出一个点的h，从h点打孔，问纸张展开后能看到几个孔。最多折叠10次，折叠有三条规则。

规则1：包含点P的面，最上层的那个被折叠。

规则2：某个面被折叠时，被折叠部分连接到的面也将被折叠。

规则3：某个面被折叠部分其上面与之有重叠的那些面也将被折叠。



分析

切割多边形，规则1直接从上往下寻找首个P点所在的面即可，难点是实现规则2和规则3。

每次被折叠产生的新面必然覆盖旧面的上方，并且原来处在更下的面翻折后将处在更上，这是面排序的依据，面数值可以按照排在越上层的索引越大来存储。

为了实现规则2和规则3，每找到一个被折叠的面，先不更新折叠后的结果，而是临时计算翻折产生的新面并根据规则2和规则3拓展寻找其他的将被折叠的面，具体来说利用状态f[1024]实现递归过程：f[i]=0表示面i暂时不需要折叠，f[i]=1表示面i需要被折叠且未对其根据规则2和规则3拓展，f[i]=2表示面i需要被折叠且已经被拓展。

实现规则2，需要用一个二位数e[1024][14]记录每个面的每条边连接到的面。

找到所有将要被折叠的面之后，再根据层级关系依次套多变形切割模板，将旧面形状更新并往面数组追加计算新产生的面，同时维护好数组e。

给一份测试数据。

更多繁琐细节，参见AC代码。

AC代码

```
#include <iostream>
#include <cstring>
using namespace std;

#define eps 1e-10
struct Point {
    double x, y;
    Point(double x = 0., double y = 0.): x(x), y(y) {}
};
typedef Point Vector;

Vector operator+ (const Vector& A, const Vector& B) {
    return Vector(A.x + B.x, A.y + B.y);
}

Vector operator- (const Vector& A, const Vector& B) {
    return Vector(A.x - B.x, A.y - B.y);
}

Vector operator* (const Vector& A, double p) {
    return Vector(A.x * p, A.y * p);
}

int dcmp(double x) {
    return abs(x) < eps ? 0 : (x < 0. ? -1 : 1);
}

double Dot(const Vector& A, const Vector& B) {
    return A.x * B.x + A.y * B.y;
}

double Cross(const Vector& A, const Vector& B) {
    return A.x * B.y - A.y * B.x;
}

bool SegmentProperIntersect(const Point& a1, const Point& a2, const Point& b1, const Point& b2) {
    double c1 = Cross(a2 - a1, b1 - a1), c2 = Cross(a2 - a1, b2 - a1);
    double c3 = Cross(b2 - b1, a1 - b1), c4 = Cross(b2 - b1, a2 - b1);
    return dcmp(c1) * dcmp(c2) < 0 && dcmp(c3) * dcmp(c4) < 0;
}

Point GetLineIntersection(const Point& P, const Vector& v, const Point& Q, const Vector& w) {
    Vector u = P - Q;
    return P + v * (Cross(w, u) / Cross(v, w));
}

bool PointInPolygon(const Point& p, const Point* poly, int n) {
    int wn = 0;
    for (int i=0; i<n; ++i) {
        int k = dcmp(Cross(poly[i+1]-poly[i], p-poly[i]));
        int d1 = dcmp(poly[i].y - p.y);
        int d2 = dcmp(poly[i+1].y - p.y);
        if (k > 0 && d1 <= 0 && d2 > 0) ++wn;
        if (k < 0 && d2 <= 0 && d1 > 0) --wn;
    }
    return wn;
}

bool overlap(const Point* p, int m, const Point* q, int n) {
    for (int i=0; i<m; ++i) for (int j=0; j<n; ++j)
        if (SegmentProperIntersect(p[i], p[i+1], q[j], q[j+1])) return true;
    for (int i=0; i<m; ++i) if (PointInPolygon(p[i], q, n)) return true;
    for (int i=0; i<n; ++i) if (PointInPolygon(q[i], p, m)) return true;
    return false;
}

Point p[1030][15], a, b, s; Vector v; int e[1030][15], c[1030], f[1030], m, n;

int find() {
    for (int i=m-1; i>=0; --i) if (PointInPolygon(a, p[i], c[i])) return i;
}

void rule(int i) {
    f[i] = 2; c[m] = 0;
    for (int j=0; j<c[i]; ++j) {
        int d = dcmp(Cross(v, p[i][j]-s));
        if (d >= 0) {
            int d2 = dcmp(Cross(v, p[i][j+1]-s));
            if (e[i][j] >= 0 && !f[e[i][j]] && (d > 0 || d2 > 0)) f[e[i][j]] = 1;
            p[m][c[m]++] = p[i][j];
            if (d > 0 && d2 < 0) p[m][c[m]++] = GetLineIntersection(p[i][j], p[i][j+1]-p[i][j], s, v);
        } else if (dcmp(Cross(v, p[i][j+1]-s)) > 0) {
            if (e[i][j] >= 0 && !f[e[i][j]] && (d > 0 || d2 > 0)) f[e[i][j]] = 1;
            p[m][c[m]++] = GetLineIntersection(p[i][j], p[i][j+1]-p[i][j], s, v);
        }
    }
    p[m][c[m]] = p[m][0];
    for (int j=i+1; j<m; ++j) if (!f[j] && overlap(p[m], c[m], p[j], c[j])) f[j] = 1;
    for (int j=0; j<m; ++j) if (f[j] == 1) rule(j);
}

Point GetProjection(const Point& p) {
    return s + v * (Dot(v, p-s) / Dot(v, v));
}

int solve() {
    p[0][0] = p[0][4] = {0., 0.}; p[0][1] = {100., 0.}; p[0][2] = {100., 100.}; p[0][3] = {0., 100.};
    m = 1; c[0] = 4; e[0][0] = e[0][1] = e[0][2] = e[0][3] = -1;
    while (n--) {
        cin >> a.x >> a.y >> b.x >> b.y; v.x = a.y-b.y; v.y = b.x-a.x; s.x = (a.x+b.x)/2.; s.y = (a.y+b.y)/2.;
        memset(f, 0, sizeof(f)); rule(find());
        for (int i=m-1, t=0; i>=0; --i) if (f[i]) f[i] = m + t++;
        for (int i=m-1; i>=0; --i) if (f[i]) {
            c[m] = 0;
            for (int j=c[i]; j>0; --j) {
                int d = dcmp(Cross(v, p[i][j]-s));
                if (d >= 0) {
                    b = GetProjection(p[i][j]); p[m][c[m]++] = Point(2*b.x-p[i][j].x, 2*b.y-p[i][j].y);
                    int d2 = dcmp(Cross(v, p[i][j-1]-s));
                    if (e[i][j-1] >= 0 && (d > 0 || d2 > 0)) e[m][c[m]-1] = f[e[i][j-1]];
                    else if (d == 0 && d2 < 0) e[m][c[m]-1] = i;
                    else e[m][c[m]-1] = -1;
                    if (d > 0 && d2 < 0)
                        e[m][c[m]] = i, p[m][c[m]++] = GetLineIntersection(p[i][j], p[i][j-1]-p[i][j], s, v);
                } else if (dcmp(Cross(v, p[i][j-1]-s)) > 0) {
                    e[m][c[m]] = e[i][j-1] >= 0 ? f[e[i][j-1]] : -1;
                    p[m][c[m]++] = GetLineIntersection(p[i][j], p[i][j-1]-p[i][j], s, v);
                }
            }
        }
        p[m][c[m]] = p[m][0]; c[++] = c[i]; c[i] = 0; p[m][c[m]] = p[i][0];
        for (int j=0; j<c[m]; ++j) p[m][j] = p[i][j], e[m][j] = e[i][j];
        for (int j=0; j<c[m]; ++j) {
            int d = dcmp(Cross(v, p[m][j]-s));
            if (d <= 0) {
                e[i][c[i]] = e[m][j]; p[i][c[i]++] = p[m][j];
                int d2 = dcmp(Cross(v, p[m][j+1]-s));
                if (d == 0 && d2 > 0) e[i][c[i]-1] = f[i];
                if (d < 0 && d2 > 0)
                    e[i][c[i]] = f[i], p[i][c[i]++] = GetLineIntersection(p[m][j], p[m][j+1]-p[m][j], s, v);
            } else if (dcmp(Cross(v, p[m][j+1]-s)) < 0)
                e[i][c[i]] = e[m][j], p[i][c[i]++] = GetLineIntersection(p[m][j], p[m][j+1]-p[m][j], s, v);
        }
        p[i][c[i]] = p[i][0];
    }
}

cin >> a.x >> a.y;
int ans = 0;
for (int i=0; i<m; ++i) if (PointInPolygon(a, p[i], c[i])) ++ans;
return ans;

int main() {
    while (cin >> n && n) cout << solve() << endl;
    return 0;
}
```